

Implementacion de Pasarela de Pago Redsys

Proyecto joyeriaTFG - Guia tecnica, seguridad, flujo de compra y pruebas en sandbox

1. Objetivo

Se ha implementado una integracion profesional con Redsys mediante redireccion al TPV Virtual. Esta modalidad evita que la aplicacion gestione datos de tarjeta, reduce riesgos de cumplimiento PCI-DSS y permite pasar de sandbox a produccion cambiando variables de entorno.

La aplicacion crea un pedido pendiente, firma la operacion, redirige al usuario al TPV y solo confirma el pago cuando Redsys envia una notificacion servidor-a-servidor con firma valida.

2. Archivos creados o modificados

Archivo	Funcion
config/redsyst.php	Centraliza entorno, FUC, terminal, clave, moneda, tipo de operacion y URLs de sandbox/produccion.
app/Services/RedsysService.php	Genera parametros, convierte importes a centimos, firma con HMAC_SHA512_V2, verifica firmas y valida respuestas.
app/Http/Controllers/RedsysController.php	Procesa la notificacion de Redsys y gestiona los retornos OK/KO del navegador.
app/Http/Controllers/CarritoController.php	Cambia el checkout para crear pedido pendiente, reservar stock y redirigir a Redsys.
resources/views/redsyst/redirect.blade.php	Formulario intermedio que envia automaticamente los campos firmados al TPV.
database/migrations/2026_05_05_170000_add_numero_pedido_redsys_t_o_PAGOS_REDSYST_table.php	Aade una referencia unica de pedido Redsys para enlazar notificaciones con pagos.
routes/web.php	Registra redsyst/notificacion, redsyst/ok y redsyst/ko.
app/Http/Middleware/VerifyCsrfToken.php	Excluye las rutas de Redsys de CSRF porque Redsys no puede enviar token Laravel.
resources/views/pedidos/index.blade.php y show.blade.php	Muestran estado de pago, referencia TPV y codigo de autorizacion.
tests/Unit/RedsysServiceTest.php	Comprueba el vector oficial de Redsys y la conversion de euros a centimos.

3. Variables de entorno

La configuracion se ha dejado preparada en .env.example. En desarrollo se usan las credenciales genericas de sandbox publicadas por Redsys.

```
REDSYS_ENVIRONMENT=sandbox
REDSYS_MERCHANT_CODE=999008881
REDSYS_TERMINAL=001
REDSYS_SECRET_KEY=sq7HjrU0BfKmc576ILgskD5srU870gJ7
REDSYS_CURRENCY=978
REDSYS_TRANSACTION_TYPE=0
```

Para produccion se cambian estas variables por las reales proporcionadas por la entidad bancaria. No hay que modificar codigo.

4. Flujo tecnico de compra

Paso	Descripcion
1	El usuario pulsa Pasar por caja desde la cesta.
2	Laravel valida que el carrito existe y que hay stock suficiente.
3	Se abre una transaccion de base de datos y se bloquean carrito/productos para evitar ventas simultaneas inconsistentes.
4	Se crea un PEDIDO en estado pendiente y sus DETALLES_PEDIDO.
5	Se reserva/descuenta stock mientras el pago queda pendiente.
6	Se crea un registro PAGOS_REDSYS pendiente con numero_pedido_redsys unico.
7	RedsysService construye Ds_MerchantParameters y Ds_Signature.
8	La vista redsys.redirect autoenvia el formulario al endpoint sandbox de Redsys.
9	Redsys procesa autenticacion y autorizacion de la tarjeta.
10	Redsys envia POST a /redsys/notificacion. Esta notificacion es la fuente de verdad.
11	La aplicacion verifica firma, importe, moneda, comercio y tipo de operacion.
12	Si todo es correcto, el pago pasa a completado. Si falla, pago error, pedido cancelado y stock restaurado.

5. Seguridad aplicada

La integracion no recoge ni almacena tarjeta, caducidad ni CVV. La tarjeta se introduce exclusivamente en el TPV de Redsys.

La confirmacion real no depende del retorno visual del usuario a /redsys/ok, sino de la notificacion firmada en /redsys/notificacion.

La firma se valida con hash_equals para evitar comparaciones inseguras. Tambien se comprueba que el importe recibido coincide con el total del pedido en centimos, que la moneda es EUR/978, que el comercio coincide con REDSYS_MERCHANT_CODE y que el tipo de transaccion es 0/autorizacion.

Las rutas Redsys estan fuera de CSRF porque reciben POST externos, pero siguen protegidas por firma criptografica. Si la firma es invalida, se responde KO y no se actualiza el pedido.

6. Firma Redsys

Redsys exige enviar tres campos: Ds_SignatureVersion, Ds_MerchantParameters y Ds_Signature. La firma implementada usa HMAC_SHA512_V2.

```
Ds_SignatureVersion = HMAC_SHA512_V2
Ds_MerchantParameters = JSON de operacion codificado en Base64 URL-safe
Ds_Signature = HMAC-SHA512 sobre Ds_MerchantParameters usando clave derivada del pedido
```

La clave derivada se obtiene cifrando el numero de pedido con AES-128-CBC y la clave secreta del comercio. Se ha anadido una prueba unitaria con el vector oficial: pedido 1234567890 y clave sq7HjrUOBfKmC576ILgskD5srU870gJ7 generan RWt3/IPTzYRMXsQtkiGRKg==.

7. Rutas implementadas

Ruta	Metodo	Uso
/redsys/notificacion	POST	Notificacion servidor-a-servidor. Es la confirmacion bancaria real.
/redsys/ok	GET/POST	Retorno visual cuando Redsys considera que el flujo ha ido bien.
/redsys/ko	GET/POST	Retorno visual cuando el usuario cancela o el pago falla.

8. Tarjetas y datos de prueba sandbox

Datos oficiales de prueba Redsys para redireccion. Solo funcionan en entorno sandbox.

Caso	Numero	Caducidad	CVV
VISA generica EMV3DS 2.2	4548 8100 0000 0003	12/49	123
Mastercard generica EMV3DS 2.1	5576 4415 6304 5037	12/49	123
Denegacion con VISA generica	4548 8100 0000 0003	12/49	999

Para simular una autorizacion correcta, usa la VISA generica con CVV 123. Para simular una operacion denegada, usa la misma tarjeta con CVV 999 y completa la autentificacion con exito en el simulador.

9. Como probar en local

Redsys necesita llamar a /redsys/notificacion desde Internet. Si la aplicacion esta solo en localhost, el retorno visual puede funcionar, pero la confirmacion servidor-a-servidor no llegara. Para una prueba real necesitas un dominio publico HTTPS, un entorno desplegado o un tunel publico hacia tu servidor local.

```
docker compose exec -T app php artisan migrate
docker compose exec -T app php artisan route:list --name=redsys
docker compose exec -T app php artisan test
```

Prueba manual recomendada: iniciar sesion, anadir producto al carrito, pasar por caja, comprobar redireccion al TPV sandbox, introducir tarjeta de prueba, volver a la aplicacion y verificar que PAGOS_REDSYS pasa de pendiente a completado.

10. Paso a produccion

Cuando el banco proporcione el TPV real, sustituye las variables de entorno y limpia cache de configuracion. La URL de produccion se selecciona automaticamente con REDSYS_ENVIRONMENT=production.

```
REDSYS_ENVIRONMENT=production
REDSYS_MERCHANT_CODE=FUC_REAL
REDSYS_TERMINAL=TERMINAL_REAL
REDSYS_SECRET_KEY=CLAVE_REAL
APP_URL=https://tu-dominio.com
```

```
php artisan config:clear
php artisan route:clear
```

Antes de abrir al publico, confirma que no queda el sandbox activo. Redsys advierte que mantener un TPV publico apuntando a pruebas permitiria pagos con tarjetas de prueba sin valor contable.

11. Verificacion realizada

Comando	Resultado
php -l sobre archivos modificados	Sin errores de sintaxis.
docker compose exec -T app php artisan migrate	Migracion aplicada correctamente.
docker compose exec -T app php artisan route:list --name=redsys	Rutas Redsys registradas.
docker compose exec -T app php artisan test	4 tests pasados.

12. Fuentes oficiales

Documentacion Redsys - Autorizacion por redireccion:

<https://pagosonline.redsys.es/desarrolladores-inicio/documentacion-operativa/autorizacion/>

Documentacion Redsys - Firma de operacion:

<https://pagosonline.redsys.es/desarrolladores-inicio/documentacion-operativa/firmar-una-operacion/>

Documentacion Redsys - Tarjetas y entornos de prueba:

<https://pagosonline.redsys.es/desarrolladores-inicio/integrate-con-nosotros/tarjetas-y-entornos-de-prueba/>